

Menu-Driven C Program for Digit-Based Operations

(using switch-case and user-defined functions)

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16
Date	9 September 2026

1. Aim

To design, develop and test a menu-driven C program that performs digit-based operations on a positive integer using the switch-case construct, where every menu option is implemented as an independent, void-returning, user-defined function that is invoked with a simple function call (no arguments passed by value or by reference).

Operations to be performed

- Check whether the digits of the number are in increasing order from left to right.
- Check whether the digits of the number are in decreasing order from left to right.
- Find the largest digit and the smallest digit present in the number.
- Find the second largest digit and the second smallest digit present in the number.

2. Algorithm

Main Program (Menu Driver)

1. Start the program.
2. Display a menu with 5 options: (1) Increasing order check, (2) Decreasing order check, (3) Largest & smallest digit, (4) Second largest & second smallest digit, (5) Exit.
3. Read the user's choice into an integer variable choice.
4. Use a switch(choice) statement to call the corresponding void function with a simple function call (e.g. checkIncreasing();).
5. If choice is invalid (not 1-5), print an error message using the default case.
6. Repeat steps 2-5 inside a do-while loop until the user selects option 5 (Exit).
7. Stop the program.

Function 1: checkIncreasing() — Increasing order check

1. Read a positive integer n.
2. Extract digits of n one at a time using $n \% 10$ and $n = n / 10$, storing them in an array (this gives digits from right to left).
3. Reverse the array so the digits are ordered from left to right.
4. Compare each digit with the next digit; if any digit is greater than the digit after it, set a flag increasing = 0 and stop comparing.
5. If the flag remains 1, print that the digits are in increasing order; otherwise print that they are not.

Function 2: checkDecreasing() — Decreasing order check

1. Read a positive integer n.
2. Extract and reverse the digits as above so they are ordered from left to right.
3. Compare each digit with the next digit; if any digit is smaller than the digit after it, set a flag decreasing = 0 and stop comparing.
4. If the flag remains 1, print that the digits are in decreasing order; otherwise print that they are not.

Function 3: findLargestSmallest() — Largest and smallest digit

1. Read a positive integer n. Initialise largest = -1 and smallest = 10.

2. Repeat while $n > 0$: extract $\text{digit} = n \% 10$; if $\text{digit} > \text{largest}$ update largest; if $\text{digit} < \text{smallest}$ update smallest; then $n = n / 10$.
3. Print the values of largest and smallest.

Function 4: findSecondLargestSmallest() — Second largest and second smallest digit

1. Read a positive integer n and extract all its digits into an array.
2. Sort the digit array in ascending order (bubble sort).
3. Remove duplicate digits to obtain a list of distinct digits.
4. If fewer than 2 distinct digits exist, report that second largest/smallest cannot be determined.
5. Otherwise, the element at index 1 of the sorted distinct list is the second smallest, and the element at index (count-2) is the second largest. Print both.

3. Test Case Table

The following test cases were prepared before coding, and later used to verify the executed output of the program (see Section 6 — Execution Output).

Menu Option 1: Increasing Order Check

S.No	Input (n)	Expected Output	Reasoning
1	12345	Digits ARE in increasing order	$1 < 2 < 3 < 4 < 5$
2	54321	Digits are NOT in increasing order	$5 > 4$, fails at first pair
3	7	Single digit — trivially increasing	Only one digit present

Menu Option 2: Decreasing Order Check

S.No	Input (n)	Expected Output	Reasoning
1	54321	Digits ARE in decreasing order	$5 > 4 > 3 > 2 > 1$
2	12345	Digits are NOT in decreasing order	$1 < 2$, fails at first pair
3	9	Single digit — trivially decreasing	Only one digit present

Menu Option 3: Largest and Smallest Digit

S.No	Input (n)	Expected Largest	Expected Smallest
1	402581	8	0
2	7	7	7

Menu Option 4: Second Largest and Second Smallest Digit

S.No	Input (n)	Distinct Digits (sorted)	2nd Smallest	2nd Largest
1	402581	0,1,2,4,5,8	1	5

S.No	Input (n)	Distinct Digits (sorted)	2nd Smallest	2nd Largest
2	1123	1,2,3	2	2
3	5555	5 (only 1 distinct digit)	N/A	N/A

4. Source Code (digit_ops.c)

The complete C program is listed below. It was compiled without warnings using: gcc -Wall -o digit_ops digit_ops.c

```
#include <stdio.h>

/* Function declarations - no parameters (simple function call) */
void checkIncreasing(void);
void checkDecreasing(void);
void findLargestSmallest(void);
void findSecondLargestSmallest(void);

int main()
{
    int choice;

    do
    {
        printf("\n==== DIGIT OPERATIONS MENU =====\n");
        printf("1. Check whether digits are in increasing order\n");
        printf("2. Check whether digits are in decreasing order\n");
        printf("3. Find the largest and smallest digit\n");
        printf("4. Find the second largest and second smallest digit\n");
        printf("5. Exit\n");
        printf("===== \n");
        printf("Enter your choice (1-5): ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                checkIncreasing();
                break;
            case 2:
                checkDecreasing();
                break;
            case 3:
                findLargestSmallest();
                break;
            case 4:
                findSecondLargestSmallest();
                break;
            case 5:
                printf("Exiting the program. Goodbye!\n");
                break;
            default:
                printf("Invalid choice! Please enter a number between 1 and 5.\n");
        }
    } while (choice != 5);

    return 0;
}
```

```

}

/* 1. Check whether digits are in increasing order from left to right */
void checkIncreasing(void)
{
    long n, temp;
    int digits[20], count = 0, i;
    int increasing = 1;

    printf("Enter a positive integer: ");
    scanf("%ld", &n);

    if (n <= 0)
    {
        printf("Invalid input! Please enter a positive integer.\n");
        return;
    }

    temp = n;
    while (temp > 0)
    {
        digits[count] = temp % 10;
        count++;
        temp = temp / 10;
    }

    /* digits[] is currently stored right-to-left; reverse to get left-to-right */
    for (i = 0; i < count / 2; i++)
    {
        int t = digits[i];
        digits[i] = digits[count - 1 - i];
        digits[count - 1 - i] = t;
    }

    for (i = 0; i < count - 1; i++)
    {
        if (digits[i] > digits[i + 1])
        {
            increasing = 0;
            break;
        }
    }

    if (count == 1)
        printf("Result: %ld has a single digit, hence trivially in increasing order.\n",
n);
    else if (increasing)
        printf("Result: The digits of %ld ARE in increasing order from left to right.\n",
n);
    else
        printf("Result: The digits of %ld are NOT in increasing order from left to
right.\n", n);
}

/* 2. Check whether digits are in decreasing order from left to right */
void checkDecreasing(void)
{
    long n, temp;
    int digits[20], count = 0, i;
    int decreasing = 1;

```

```

printf("Enter a positive integer: ");
scanf("%ld", &n);

if (n <= 0)
{
    printf("Invalid input! Please enter a positive integer.\n");
    return;
}

temp = n;
while (temp > 0)
{
    digits[count] = temp % 10;
    count++;
    temp = temp / 10;
}

for (i = 0; i < count / 2; i++)
{
    int t = digits[i];
    digits[i] = digits[count - 1 - i];
    digits[count - 1 - i] = t;
}

for (i = 0; i < count - 1; i++)
{
    if (digits[i] < digits[i + 1])
    {
        decreasing = 0;
        break;
    }
}

if (count == 1)
    printf("Result: %ld has a single digit, hence trivially in decreasing order.\n",
n);
else if (decreasing)
    printf("Result: The digits of %ld ARE in decreasing order from left to right.\n",
n);
else
    printf("Result: The digits of %ld are NOT in decreasing order from left to
right.\n", n);
}

/* 3. Find the largest digit and the smallest digit */
void findLargestSmallest(void)
{
    long n, temp;
    int largest = -1, smallest = 10, digit;

    printf("Enter a positive integer: ");
    scanf("%ld", &n);

    if (n <= 0)
    {
        printf("Invalid input! Please enter a positive integer.\n");
        return;
    }

    temp = n;
    while (temp > 0)

```

```

    {
        digit = temp % 10;
        if (digit > largest)
            largest = digit;
        if (digit < smallest)
            smallest = digit;
        temp = temp / 10;
    }

    printf("Result: Largest digit of %ld = %d\n", n, largest);
    printf("Result: Smallest digit of %ld = %d\n", n, smallest);
}

/* 4. Find the second largest digit and second smallest digit */
void findSecondLargestSmallest(void)
{
    long n, temp;
    int digits[20], count = 0, i, j;
    int unique[20], ucount = 0;

    printf("Enter a positive integer: ");
    scanf("%ld", &n);

    if (n <= 0)
    {
        printf("Invalid input! Please enter a positive integer.\n");
        return;
    }

    temp = n;
    while (temp > 0)
    {
        digits[count] = temp % 10;
        count++;
        temp = temp / 10;
    }

    /* sort digits in ascending order using bubble sort */
    for (i = 0; i < count - 1; i++)
    {
        for (j = 0; j < count - 1 - i; j++)
        {
            if (digits[j] > digits[j + 1])
            {
                int t = digits[j];
                digits[j] = digits[j + 1];
                digits[j + 1] = t;
            }
        }
    }

    /* remove duplicate digits */
    for (i = 0; i < count; i++)
    {
        if (i == 0 || digits[i] != digits[i - 1])
        {
            unique[ucount] = digits[i];
            ucount++;
        }
    }
}

```

```

    if (ucount < 2)
    {
        printf("Result: %ld does not have enough distinct digits for second
largest/smallest.\n", n);
        return;
    }

    printf("Result: Second smallest digit of %ld = %d\n", n, unique[1]);
    printf("Result: Second largest digit of %ld = %d\n", n, unique[ucount - 2]);
}

```

5. Compilation, Execution & Actual Output

Compilation command used:

```

$ gcc -Wall -o digit_ops digit_ops.c
(compiled successfully, no errors or warnings)

```

The program was run and fed the inputs from the test case table above (menu choices 1-4 in sequence, followed by 5 to exit). The actual console output obtained is reproduced below:

```

===== DIGIT OPERATIONS MENU =====
1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit
=====
Enter your choice (1-5): Enter a positive integer: Result: The digits of 12345 ARE in
increasing order from left to right.

===== DIGIT OPERATIONS MENU =====
1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit
=====
Enter your choice (1-5): Enter a positive integer: Result: The digits of 54321 are NOT in
increasing order from left to right.

===== DIGIT OPERATIONS MENU =====
1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit
=====
Enter your choice (1-5): Enter a positive integer: Result: 7 has a single digit, hence
trivially in increasing order.

===== DIGIT OPERATIONS MENU =====
1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit

```

```
=====
```

```
Enter your choice (1-5): Enter a positive integer: Result: The digits of 54321 ARE in decreasing order from left to right.
```

```
===== DIGIT OPERATIONS MENU =====
```

1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit

```
=====
```

```
Enter your choice (1-5): Enter a positive integer: Result: The digits of 12345 are NOT in decreasing order from left to right.
```

```
===== DIGIT OPERATIONS MENU =====
```

1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit

```
=====
```

```
Enter your choice (1-5): Enter a positive integer: Result: 9 has a single digit, hence trivially in decreasing order.
```

```
===== DIGIT OPERATIONS MENU =====
```

1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit

```
=====
```

```
Enter your choice (1-5): Enter a positive integer: Result: Largest digit of 402581 = 8  
Result: Smallest digit of 402581 = 0
```

```
===== DIGIT OPERATIONS MENU =====
```

1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit

```
=====
```

```
Enter your choice (1-5): Enter a positive integer: Result: Largest digit of 7 = 7  
Result: Smallest digit of 7 = 7
```

```
===== DIGIT OPERATIONS MENU =====
```

1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit

```
=====
```

```
Enter your choice (1-5): Enter a positive integer: Result: Second smallest digit of 402581 = 1  
Result: Second largest digit of 402581 = 5
```

```
===== DIGIT OPERATIONS MENU =====
```

1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit

```
=====
```

```

Enter your choice (1-5): Enter a positive integer: Result: Second smallest digit of 1123 =
2
Result: Second largest digit of 1123 = 2

===== DIGIT OPERATIONS MENU =====
1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit
=====
Enter your choice (1-5): Enter a positive integer: Result: 5555 does not have enough
distinct digits for second largest/smallest.

===== DIGIT OPERATIONS MENU =====
1. Check whether digits are in increasing order
2. Check whether digits are in decreasing order
3. Find the largest and smallest digit
4. Find the second largest and second smallest digit
5. Exit
=====
Enter your choice (1-5): Exiting the program. Goodbye!

```

6. Result

The menu-driven C program was successfully designed, coded, compiled and executed. All four digit-based operations — increasing order check, decreasing order check, largest/smallest digit, and second largest/second smallest digit — were implemented as separate void, no-argument, user-defined functions and invoked through a switch-case menu.

On comparing the actual output (Section 5) against the expected output prepared in the test case table (Section 3), all test cases matched successfully. Hence, the program works correctly for all the prepared use cases, including edge cases such as single-digit numbers and numbers with repeated digits.